

Redistribute neighborhood in the Pick-up and Delivery with Cooperative Robots

Chi Thao Nguyen¹, Alain Quilliot¹, Jean-Philippe Gayon¹,
Viet Hung Nguyen¹, Anh Son Ta²

¹LIMOS, Université Clermont-Auvergne, 1 rue de la Chebarde,
Aubiere, 63170 France.

²Faculty of Mathematics and Informatics, Hanoi University of Science
and Technology, No.1 Dai Co Viet Street, Hanoi, 100000, Vietnam.

Contributing authors: chi_thao.nguyen@uca.fr; alain.quilliot@uca.fr;
j-philippe.gayon@uca.fr; viet_hung.nguyen@uca.fr;
son.taanh@hust.edu.vn;

Abstract

This study addresses the Pickup and Delivery Problem with Cooperative Robots (PDP-CR), which optimizes the cooperation of multiple robots to complete pickup and delivery tasks in a warehouse setting. Tasks are characterized by their root requirements and time constraints, and synchronization among robots to execute them. This paper proposes a novel neighborhood for the PDP-CR based on redistributing flows within the solution graph. A tabu search algorithm incorporating the new neighborhood is developed and compared against other approaches. Results demonstrate the effectiveness of the proposed neighborhood, yielding improvements in objective value and competitiveness in solving time.

Keywords: Pick-up and delivery with cooperative robots, Metaheuristics, neighborhood, RCPSP with transfer time

Todo

- Introduction
- Update numerical results? Table how much improvement compared to the starting solution.
- Do we need to compare with the exact method?

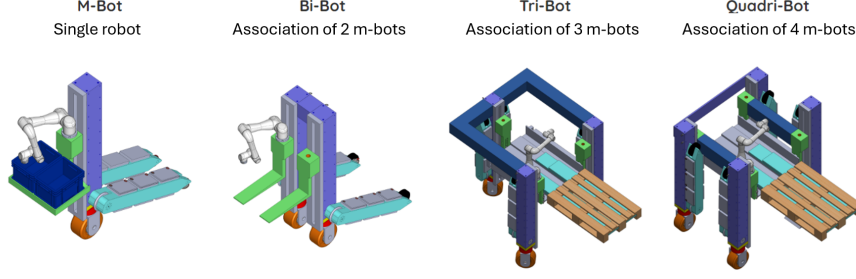


Fig. 1 Reconfiguration of warehouse robots, from <https://www.mecabotix.com/>

- Complexity of dedicated algorithm.
- State how to break tie topological order
- How new compared to conference results?

1 Introduction(obsoleted)

In the Pickup and Delivery Problem with Cooperative Robots (PDP-CR), we manage a fleet of several identical robots to complete a set of pickup and delivery tasks inside a warehouse. Each task requires a specified number of robots and involves transporting goods from a pick-up point to a destination. For example, a package inside the warehouse may be too large for a single robot to handle. In this case, multiple robots need to cooperate to complete the task. Tasks cannot begin until all required robots are present at the pick-up location. A task is considered done if all robots needed for the task reach its pick-up point and then go to the destination simultaneously. After finishing a task, the robots can be assigned to other ones. All robots start their routes at a starting depot and finish at a return depot. The goal of PDP-CR is to minimize the makespan, i.e. the time at which the last robot returns to the depot.

At a glance, one can find that PDP-CR has some similarities with other problems.

The Vehicle Routing Problem (VRP) [1], one of the most important combinatorial optimization challenges, focuses on efficiently distributing goods or services to customers using a fleet of vehicles, typically starting from a central depot. The Pickup Delivery Problem (PDP) [2, 3] involves transporting goods from an origin to a destination, and its mathematical model is a variant of the VRP with added constraints to ensure certain locations are visited before others. Synchronization in VRP [4–7] refers to the constraints that require two or more vehicles to complete a single task. For example, the synchronization between multiple homecare staff, they must be simultaneously present at a customer [5] or synchronization between the mailman crews and the vehicles [7]. PDP-CR can be seen as a special case of the Resource Constrained Project Scheduling Problem (RCPSP) with transfer time where there is only one resource and no precedence constraint. We can see the robots as a type of resource, and every task in PDP-CR is a job in RCPSP with transfer time.

The primary goal of this paper is to propose an efficient neighborhood for the Pickup and Delivery Problem with Cooperative Robots (PDP-CR). The remainder of this paper is organized as follows: Section 2 formalizes the PDP-CR, and Section 3

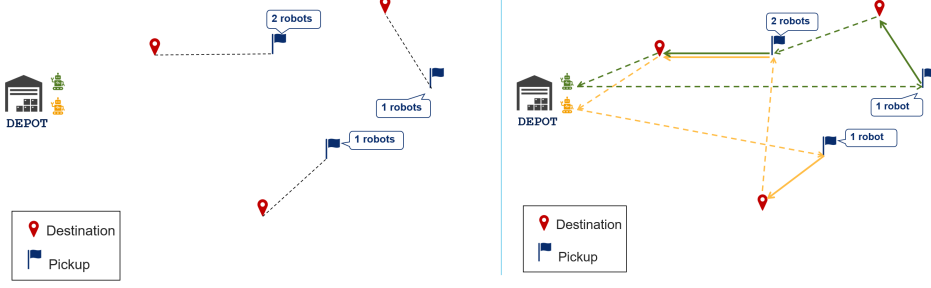


Fig. 2 An example with 2 starting robots. Left: the layout and right: the solution

explains the proposed redistribute neighborhood and details other neighborhoods from the literature. Section 4 describes the tabu search algorithm and how we integrate the redistribute neighborhood into it, while Section 5 presents computational results. Finally, Section 6 concludes with insights and future research directions.

2 Problem Definition

We have a fleet M of m identical robots and a set N of n tasks. Every task i is characterized by its pick-up location p_i , destination d_i , processing time c_i , and number of robots required r_i . A task i cannot start until r_i robots arrive at its pickup point. This implies that the early robots that visit the task i must wait for all the robots required to come. As soon as there are r_i robots at the pick-up point of the task, they process the task from its pick-up point p_i and reach d_i . This takes c_i time units. It is assumed that all the robots start their paths at a starting depot, denoted by 0, and have to finish their paths at a returning depot, denoted by $n+1$. They both have process time $c_0 = c_{n+1} = 0$. Let $N_{all} = \{0, 1, \dots, n, n+1\}$ contains all the tasks as well as the two depots. $N_0 = \{0, 1, \dots, n\}$ contains the starting depot and all the tasks. Similarly, $N_* = \{1, \dots, n+1\}$ contains all the tasks and the returning depot. t_{ij} is the time for any robot to move from task i to task j on the graph, corresponding to the time a robot moves from the destination d_i of task i to the pick-up point p_j of task j .

The objective of the Pick-up and Delivery Problem with Cooperative Robots is to minimize the makespan, i.e., when the last robot reaches the return depot. We assume that there are no conflicts involved while the robots are travelling, and the robots can work for infinite time without the need of recharging the battery.

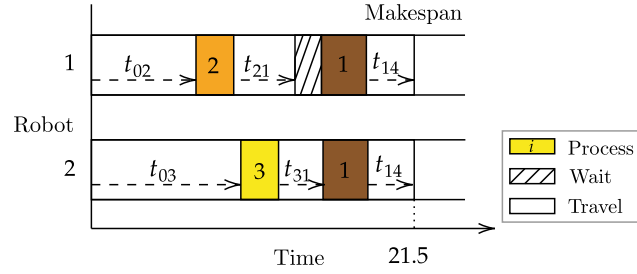
Example 1 We consider a small PDP-CR instance consisting of $n = 3$ tasks and $m = 2$ robots. The coordinates of the pick-up points and delivery locations, processing times, and number of required robots for each task are given in Table 1. Traveling time between tasks is given in Table 2. In this example, the starting depot is also the returning depot. The layout for this example is roughly illustrated in Figure 4 (left). In the figure, task 1 is the horizontal task in the middle, task 2 is the diagonal task at the bottom, and the remaining one is task 3. A feasible solution for this instance is depicted by the Gantt chart in Figure 3, and the paths of the robots in this solution are also illustrated in Figure 4 (right). One can observe that after the two robots finish task 2 and task 3, they will have to travel to the pick-up

Table 1 Data for the problem in example 1

i	p_i	d_i	r_i	c_i
0	(0,3)	(0,3)	-	0
1	(6,4)	(3,4)	2	3
2	(7,2)	(5,0)	1	2.5
3	(10,3)	(9,5)	1	2.5
4	(0,3)	(0,3)	-	0

Table 2 Travelling time between tasks t_{ij} in example 1

$i \setminus j$	1	2	3	4
0	6	7	10	0
1	-	2	7	3
2	4	-	6	6
3	3	4	-	9

**Fig. 3** Gantt chart representation of a solution in example 1

point of task 1. Since the first robot arrived before the second one, it must wait for robot 2. The waiting time is represented by the block with diagonal lines in the Gantt chart. After the second robot reaches task 1, they process the task together, and then they arrive at the returning depot together. So at some point, the robots may share the same route to perform a task or on the way to another task that needs multiple robots.

MILP formulations for the PDP-CR have been proposed in [8] and [9], namely the VRP-based and flow-based formulations. In the following, we describe the flow-based model from [9]. We consider a robot as a flow unit. Let $x_{ij} = 1$ if there are a robot do task j right after task i , $f_{ij} \in \mathbb{R}$ denotes the number of flow going to task j right after i . $z_i \in \mathbb{R}$ denotes the time robots start to work task i . If $i = n + 1$ then z_{n+1} is the time all robots return to the depot. And z_0 is the time robots leave the starting depot. And let $N^+ = \{0\} \cup N$ and $N^- = N \cup \{n + 1\}$. The flow-based model to minimize

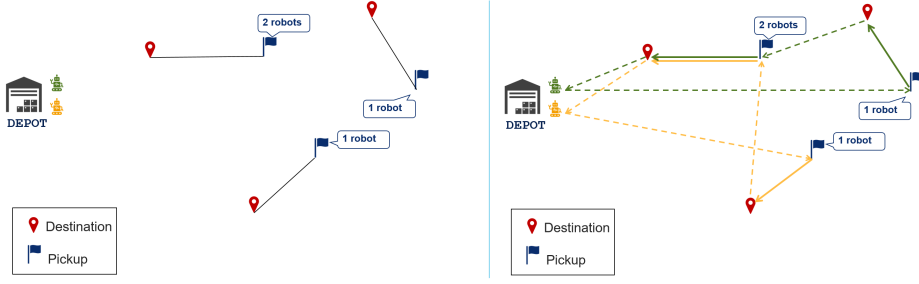


Fig. 4 A visualization of example 1. On the left: the layout; right: path of the robots

the makespan of PDP-CR is as follow:

$$\min \quad z_{n+1} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in N^-} f_{0j} = m, \quad (2)$$

$$\sum_{i \in N^+} f_{i,n+1} = m, \quad (3)$$

$$\sum_{i \in N^+} f_{ij} = r_i \quad \forall j \in N \quad (4)$$

$$\sum_{j \in N^-} f_{ij} = r_i \quad \forall i \in N \quad (5)$$

$$z_0 = 0, \quad (6)$$

$$z_j + M(1 - x_{ij}) \geq z_i + c_i + t_{ij} \quad \forall i \in N^+, \forall j \in N^-, i \neq j, \quad (7)$$

$$f_{ij} \geq x_{ij} \quad \forall i \in N^+, \forall j \in N^-, i \neq j, \quad (8)$$

$$r_i x_{ij} \geq f_{ij} \quad \forall i \in N^+, \forall j \in N^-, i \neq j, \quad (9)$$

$$f_{ij} \in \mathbb{R} \quad \forall i \in N^+, \forall j \in N^-, i \neq j \quad (10)$$

$$z_i \in \mathbb{R} \quad \forall i \in N^- \quad (11)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in N^+, \forall j \in N^-, i \neq j \quad (12)$$

In the formulation, the objective function (1) calls for minimizing the time that all robots reach the returning depot z_{n+1} , which is the makespan. Constraints (2) and (3) ensure that there are m flow units leaving the starting depot and m flow units reach the returning depot. (4) and (5) is the flow conversation constraint for each task. The starting time of all flows leaving the source is 0 is guaranteed by constraint (6). The constraint (7) is a Subtour Elimination Constraint (SEC) and is the linearization of the following constraint:

$$x_{ij} = 1 \implies z_j \geq z_i + c_i + t_{ij} \quad \forall i \in N^+, \forall j \in N^-, i \neq j$$

The above constraint implies that if there are robots travel to task j right after finishing task i , then the starting time of task j must be at least the finishing time of task i ($z_i + c_i$) plus the traveling time from task i to task j . This will make sure task j cannot start until all of the robots are available at the starting point of j . The value of M is obtained using Algorithm 1. And finally, constraints (8) and (9) imply the relation between f and x .

3 The neighborhoods

3.1 Redistribute neighborhood

In this section, we propose a novel neighborhood for a solution of PDP-CR based on the idea of cutting and redistributing flows across a solution graph. To describe our neighborhood, we will adapt the solution representation as a flow graph used in [10], where the authors used for the RCPSP. A solution of PDP-CR could be described by a Directed Acyclic Graph $\mathcal{F}$ where every node is a task. For every arc (i, j) , we know the number of flows (a flow unit representing a robot) traversing the arc f_{ij} and the traveling time t_{ij} . We know or can calculate the following information for every task i in N .

Algorithm 1 Calculate L_j

Input:

- Set of tasks N and their process time information.
- Traveling time matrix between tasks.
- A solution represented by the DAG $\mathcal{F}$.

Output: The earliest time of L_j of every task j

```

1:  $TL \leftarrow$  topological ordering of  $\mathcal{F}$ 
2:  $L_0 \leftarrow 0$ 
3:  $L_j \leftarrow -\infty$  for all  $j \in TL \setminus \{0\}$ 
4: for each task  $i$  in  $TL$  do
5:   for each successor  $j$  of  $i$  do
6:      $L_{cur} \leftarrow L_i + c_i + t_{ij}$ 
7:     if  $L_j < L_{cur}$  then
8:        $L_j \leftarrow L_{cur}$ 
9:     end if
10:  end for
11: end for
```

- processing time c_i .
- L_i : the earliest time of task j can start doing, calculate by applying algorithm 1.
- $\bar{L}_i = L_i + c_i$: the earliest finish time of task i .
- L_i^* : the time of a longest path from pickup point of i to the sink, calculate by applying algorithm 2.

Algorithm 2 Calculate L_j^*

Input:

- Set of tasks N and their process time information.
- Traveling time matrix between tasks.
- A solution represented by the DAG $\mathcal{F}$.

Output: The time of longest path from every task j to the sink L_j .

```
1:  $TL \leftarrow$  reverse topological ordering of  $\mathcal{F}$ 
2:  $L_{n+1} \leftarrow 0$ 
3:  $L_j \leftarrow -\infty$  for all  $j \in TL \setminus \{n+1\}$ 
4: for each task  $j$  in  $TL$  do
5:   for each predecessor  $i$  of  $j$  do
6:      $L_{cur}^* \leftarrow L_j^* + c_i + t_{ij}$ 
7:     if  $L_i^* < L_{cur}^*$  then
8:        $L_i^* \leftarrow L_{cur}^*$ 
9:     end if
10:  end for
11: end for
```

We have the *topology ordering* of the graph $\mathcal{F}$ represents the current solution. A topology ordering is a linear ordering of the tasks such that for every arc (i, j) in $\mathcal{F}$, i is before j in the ordering. Then we can perform a cut at $\zeta \in [1, |N|]$ on this ordering and split the graph into two parts. The first part contains ζ tasks on the left of the cut, we call this set N^{left} , note that N^{left} always contains the source. The second part includes $(|N| - \zeta)$ tasks on the right of the cut, we call this set N^{right} . This set always contains the sink. Currently, ζ is chosen randomly.

The graph is now disconnected. To improve the makespan, we need to re-distribute the flows between the tasks in N^{left} and N^{right} . With the cut, associating with each task i in N^{left} , we denote $\pi_i^{left} = \sum_{j \in N^{right}} f_{ij}$ the number of flows leaving i and go to at least one of the task in N^{right} . And similarly, π_j^{right} is the number of flows entering tasks j in N^{right} from N^{left} . If we connect i from N^{left} to j from N^{right} the makespan will be at least equal to $\bar{L}_i + t_{ij} + L_j^*$. To find the best solution of the current cut, we solve the following:

$$(RP) \quad \min \quad C_{max} \quad (13)$$

$$\text{s.t.} \quad (\bar{L}_i + t_{ij} + L_j^*)x_{ij} \leq C_{max}, \quad \forall i \in N^{left}, \forall j \in N^{right} \quad (14)$$

$$\sum_{j \in N^{right}} f_{ij} = \pi_i^{left}, \quad \forall i \in N^{left} \quad (15)$$

$$\sum_{i \in N^{left}} f_{ij} = \pi_j^{right}, \quad \forall j \in N^{right} \quad (16)$$

$$r_i x_{ij} \geq f_{ij} \quad \forall i \in N^{left}, \forall j \in N^{right} \quad (17)$$

$$f_{ij} \geq x_{ij} \quad \forall i \in N^{left}, \forall j \in N^{right} \quad (18)$$

Example 2 To further understand the neighborhood, let's consider the example figure below. Figure 5 depicts a solution represented by a flow graph, as well as the number of flows (robots) traveling the edges. Information associated with nodes and travelling time are also presented. The earliest finishing time of each task and time of a longest path from pick up point of i to the sink are calculated in Figure 6. We cut the topological ordering at $\zeta = 4$, split the flow graph into $N^{left} = \{S, A, B, C\}$ and $N^{right} = \{D, E, S'\}$ (Figure 7). To obtain a solution corresponding to $\zeta = 4$, we solve (RP) with information of N^{left} and N^{right} in Figure 8 and reconnect the flow graph with its solution.

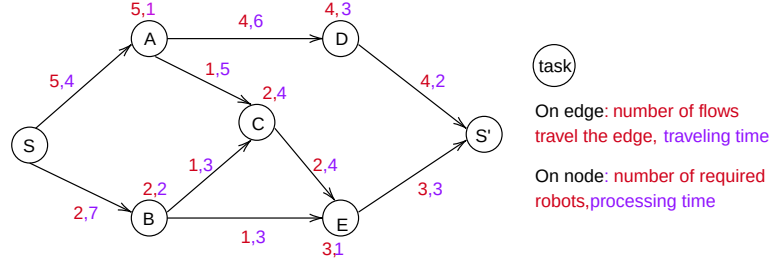


Fig. 5 Informations on arcs and nodes of a solution depicted by a flow graph

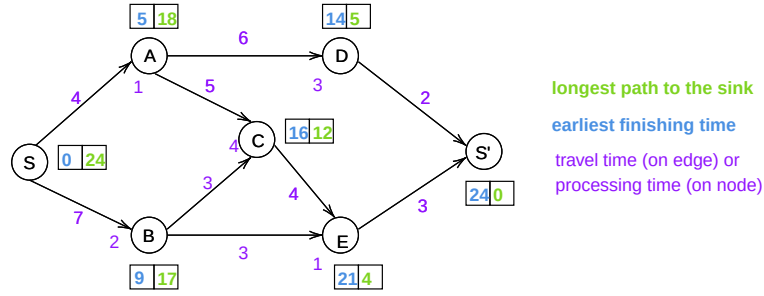


Fig. 6 Flow graph with earliest starting time of the nodes and longest path to sink calculated

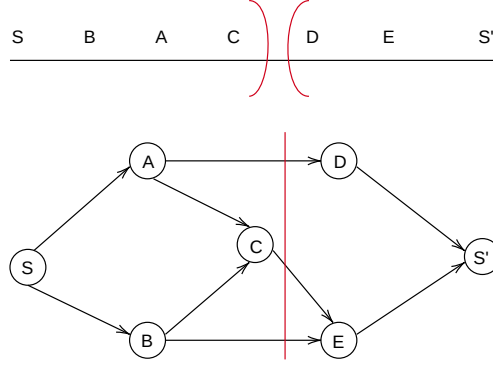


Fig. 7 A cut with $\zeta = 4$ between task C and D on a topology ordering and the corresponding cut on the flow graph

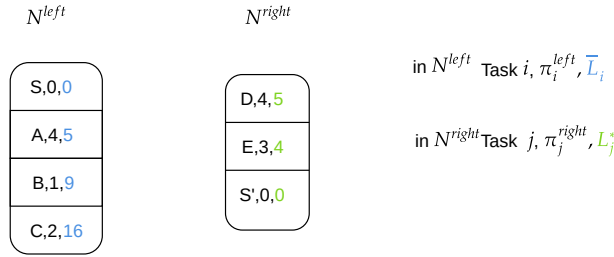


Fig. 8 The sets after the cut

Algorithm 3 Redistribute flows

Input:

- The current solution graph and its corresponding C_{max} .
- 2 task sets N^{left} and N^{right} .
- For each task i in N^{left} : $\pi_i^{left}, \bar{L}_i$.
- For each task j in N^{right} : π_j^{right}, L_j^* .
- Traveling time matrix between tasks.

Output: A solution graph after flows have been redistributed across the cut.

- 1: $\Delta_{ij} \leftarrow \bar{L}_i + t_{ij} + L_j^*$ for each i in N^{left} , j in N^{right}
 - 2: $\text{Stop} \leftarrow \text{False}$
 - 3: **while** Stop is **False** **do**
 - 4: $C_{max} \leftarrow \min\{C_{max}, \max_{(i,j): f_{ij} \neq 0} \{\Delta_{ij}\}\}$
 - 5: $A^f = \{(i, j) | i \in N^{left}, j \in N^{right}, \Delta_{ij} \geq C_{max}\}$ $\triangleright A^f$: Set of forbidden arcs
 - 6: $A^{max} \leftarrow \{(i, j) \in A^f | f_{ij} \neq 0\}$
 - 7: Consider randomly an arc (i^*, j^*) in A^{max} $\triangleright (i^*, j^*)$ is the arc we want to remove
 - 8: Search for a sequence $\Gamma = (i^*, j_1, i_2, j_2, \dots, j_{K-1}, i_K, j^*)$ such that:
 - For any k : i_k in N^{left} , j_k in N^{right}
 - For any k : $f_{i_{k+1}, j_k} \neq 0$.
 - Neither (i^*, j_1) or (i_K, j^*) in A^f
 - For any $2 \leq k \leq K-1$: (i_k, j_k) not in A^f
 - 9: **if** search fail **then**
 - 10: Stop, return the solution graph
 - 11: **else**
 - 12: $\delta \leftarrow \min\{f_{i^* j^*}, \min_{k \leq K} \{f_{i_{k+1}, j_k}\}\}$
 - 13: Modified the flows on the solution graph:
 - $f_{i^* j^*} \leftarrow f_{i^* j^*} - \delta$
 - for all $k \leq K$: $f_{i_{k+1}, j_k} \leftarrow f_{i_{k+1}, j_k} - \delta$
 - for all $2 \leq k \leq K-1$: $f_{i_k j_k} \leftarrow f_{i_k j_k} + \delta$
 - $f_{i^* j_1} \leftarrow f_{i^* j_1} + \delta, f_{i_K j^*} \leftarrow f_{i_K j^*} + \delta$
 - 14: **end if**
 - 15: **end while**
-

Table 3 Additional info
 t_{ij} and Δ_{ij}

$i \setminus j$	t_{ij}		Δ_{ij}	
	D	E	D	E
A	10	4	20	17
B	2	3	16	16
C	1	4	22	24

In order to solve (*RP*) effectively, we propose a (polynomial) Algorithm 3. The idea of the algorithm is to decrease the makespan of a solution by removing the critical arc (i^*, j^*) across the cut and finding alternative successors for i^* as well as finding alternative predecessors for j^* .

In line 1, Δ_{ij} is the length of paths crossing (i, j) if we connect i from N^{left} to j from N^{right} . We iterate the problem until no further improvement can be made, indicated by the variable "Stop" (line 3). In line 4 we update C_{max} if the new value of the longest path in the current solution is smaller than the current C_{max} . We also forbid paths that have their length not shorter than the current C_{max} in line 6. This algorithm's improvement of C_{max} comes from line 8 and 13. We want to remove the arc (i^*, j^*) , which is the arc on one of the critical paths of the current solution, and find alternative successors to i and alternative predecessors to j and to make sure the flow conservation constraint is held for all the associated nodes.

Example 3 Consider the above example depicted from Figure 5 to Figure 8. The cut is between C and D as illustrated in figure 7. In Table 3, we give additional information about traveling time t_{ij} and calculated Δ_{ij} between arcs in N^{left} and N^{right} . The current C_{max} is 24 (Figure 6). The current flow value between tasks across the cut is depicted in Figure 9(a). Start iterating, we have $A^f = \{(C, E)\}$, $A^{max} = \{(C, E)\}$, so (i^*, j^*) is now (C, E) . We search for a feasible sequence Γ depicted in line 8 and found $\Gamma = (C, D, A, E) = (i^*, j_1, i_2, j^*)$. $\delta = \min\{f_{i^*j^*}, f_{i_2j_1}\} = \min\{f_{CE}, f_{AD}\} = \min\{4, 2\} = 2$. We redistribute the flows as follow: $f_{CE} = 2 - 2 = 0$, $f_{AD} = 4 - 2 = 2$, $f_{CD} = 0 + 2 = 2$, $f_{AE} = 0 + 2 = 2$. (Figure 9(b)). Figure 9(c) depicts the new flows across the cut. Move to the next iteration, we update $C_{max} = \min\{C_{max}, \max\{\Delta_{AD}, \Delta_{AE}, \Delta_{CD}, \Delta_{BE}\}\} = \min\{24, \max\{20, 17, 22, 16\}\} = 22$. $A^f = \{(C, E), (C, D)\}$, $A^{max} = \{(C, D)\}$. (i^*, j^*) is now (C, D) . We cannot find any feasible Γ from C to D , so the algorithm terminates, the flow values across the cut after being redistributed are illustrated in Figure 9(d). Returns the current solution graph.

3.2 Reroute, reverse neighborhoods

This section will quickly describe the reroute neighborhood and the reverse neighborhood presented in [10], which are used complementary with our redistribute neighborhood.

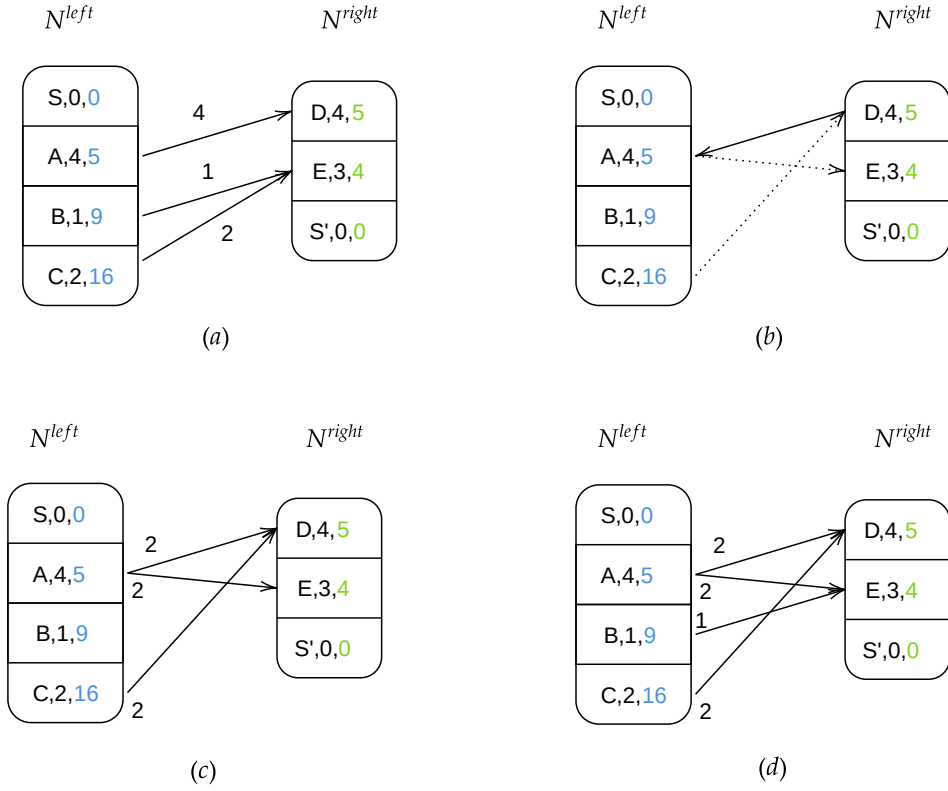


Fig. 9 (a) Initial flows value across the cut
(b) A feasible sequence $\Gamma = \{C, D, A, E\}$
(c) Flow value of tasks associating with Γ after being redistributed
(d) Flow values across the cut after redistributed

Reroute neighborhood $\mathcal{N}_{reroute}$ The general idea of reroute neighborhood is to swap the flows between 2 arcs. Concretely, two arcs (i, j) and (u, v) with $f_{ij} > 0$ and $f_{uv} > 0$ are selected. Then, an amount of $q \in \{1, \dots, \min\{f_{ij}, f_{uv}\}\}$ units of flow is rerouted from task i to task v as well as from task u to task j . This results the new graph with the following modifications $f'_{ij} = f_{ij} - q$, $f'_{uv} = f_{uv} - q$, $f'_{iv} = f_{iv} + q$ and $f'_{vj} = f_{vj} + q$ as illustrated on Figure 10.

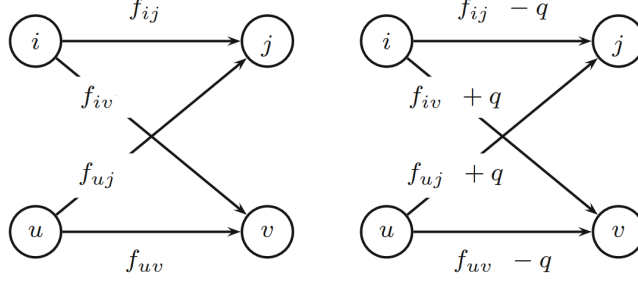


Fig. 10 Flows before (left) and after (right) a reroute modification have been applied, adapted from [10]

In order to reduce the size of the neighborhoods, we introduce the neighborhood $\mathcal{N}_{reroute}^{max,ca}$, in which one critical arc (i, j) and an arbitrary arc (u, v) are selected instead of two arbitrary arcs, and the maximal amount of $q = \min\{f_{ij}, f_{uv}\}$ units of flows is rerouted. This neighborhood can be further reduced to a neighborhood $\mathcal{N}_{reroute}^{max,ca,\delta}$ by only considering one arc (i, j) with the largest traveling time t_{ij} among critical arcs.

Reverse neighborhood $\mathcal{N}_{reverse}$ The idea of the reverse neighborhood is to reverse the direction of flow (i, j) on the graph flow and a series of modifications of its inflows (i.e. the flows enter task i : (u, i)) and outflows (i.e. the flows leaving j : (j, v)). In detail:

Set U of tasks are selected based on a priority rule such that arc (u, i) with $f_{ui} > 0$ exists between task u as i and

$$\sum_{u \in U} f_{ui} \geq f_{ij}$$

holds. The set are chosen such that no activity $u \in U$ can be removed from the set U without violating this inequality (i.e., the inequality does not hold for any proper subset of the set U). Similarly, set V are chosen based on a priority rule such that arc (j, v) with $f_{jv} > 0$ exists between task j and $v \in V$ and

$$\sum_{v \in V} f_{jv} \geq f_{ij}$$

holds. Again, the set V is chosen such that no activity $v \in V$ can be removed from the set V without violating this inequality. In the following, we assume that the subset U contains a tasks $u_1, \dots, u_a$, while subset V contains total of b activities $v_1, \dots, v_b$. Now, a reverse move first reverses the direction of arc (i, j) such that an amount of $f'_{ji} = f_{ij}$ flows move from task j to task i in the resulting resource flow $\mathcal{F}'$. Hence, the flows $f_{ui}, u \in U$ and $f_{jv}, v \in V$ also must be modified to ensure the flows conservation constraints. Specifically, all $f_{u_\alpha i}$ flows are redirected from $u_\alpha \in U$ with $\alpha = 1, \dots, a-1$ to activity j . Additionally,

$$q_a = f_{ij} - \sum_{\lambda=1}^{a-1} f_{u_\lambda i}$$

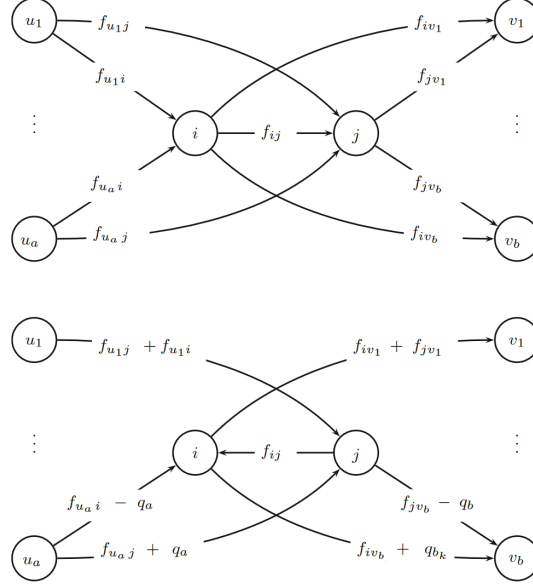


Fig. 11 Flows before (upper) and after (below) a reverse modification have been applied, adapted from [10]

flows are redirect from remaining task u_a to task j .

Similarly, all f_{jv_γ} flows are redirected from i to activities $v_\gamma \in V$ with $v = 1, \dots, b-1$, while

$$q_b = f_{ij} - \sum_{\gamma=1}^{b-1} f_{jv_\gamma}$$

flows redirected from i to v_b (Figure 11). It should be noted that the last activities u_a and v_b to be considered are also the last tasks that have been selected by the priority rule.

To further reduce the neighborhood, $\mathcal{N}_{reverse}^{ca}$ in which only critical arcs are selected to apply reverse moves. The current priority rule to choose set U of reversing arc (i, j) is to select tasks according to non-decreasing earliest arrival times of the flows (or robots) at the receiving task i . To choose set V , the rule is selects tasks based on non-decreasing traveling times between j and its successors on $\mathcal{F}$.

4 A tabu search algorithm

While our neighborhood are flexible and can be integrated into multiple metaheuristics schema, we choose to implement a tabu search algorithm based on [10] we want to have a direct comparison between our method vs an existing one in the literature, note that the problem in [10] are RCPSP, and hence, we have to make some slight adjustment to fit ours. The tabu search at a glance is as follow:

- Initialize:

- Find an initial solution.
- Set it as the current and the best solution.
- Initialize an empty tabu list.
- While not stop condition:
 - **Generate candidates from the neighborhoods of the current solution.**
 - Evaluate non-tabu candidates.
 - Set the best candidate as the current solution. Update the tabu list.
 - Update the best solution if possible.
 - Remove the oldest element from tabu list if needed.

First, we want to find an initial solution. The tabu list contains the list In each iteration of the tabu search, a solution is selected from the neighborhood $\mathcal{N}_1 = \mathcal{N}_{redis} \cup \mathcal{N}_{reroute}^{max,ca,\delta} \cup \mathcal{N}_{reverse}^{ca}$. Then, we evaluate non-tabu neighbors. If a solution with a better objective function value than the current solution is found, this solution is immediately accepted. Otherwise, if the best non-tabu solution is from the neighborhood $\mathcal{N}_{redis}$, we choose it with probability β and choose the best non-tabu solution from the neighborhood $\mathcal{N}_{reroute}^{max,ca,delta} \cup \mathcal{N}_{reverse}^{ca}$ is chosen with probability $(1 - \beta)$, even if the objective is worse than the current one. The purpose of β is to nudge the algorithm to explore in neighborhoods other than $\mathcal{N}_{redis}$. Because reroute and reverse tend to make the objective worse; but redistribute, in the worst case, the objective value stay the same. If we choose the best non-tabu neighbor at each iteration, it is likely from the redistribute neighborhood and hence limits the exploration of the search.

After a solution has been chosen, the tabu lists are updated based on the selected modification. We use two tabu list TL_{add} and TL_{redis} . If a reroute move has been applied to arcs (i, j) and (u, v) , we store (i, j) and (u, v) in the TL_{add} . Similarly, if a reverse modification has been applied to reverse arc (i, j) , we store (i, j) in TL_{add} . A reroute move between (i', j') and (u', v') is tabu if (i', u') or (u', j') is in TL_{add} . Similarly, a reverse move on (i', j') is considered tabu if (j', i') is in TL_{add} . If a redistribute move has been applied at cut ζ , we store ζ in TL_{redis} . A redistribute move is considered tabu if ζ is in TL_{redis} . The tabu lists are able to avoid cycling (i.e., the revisiting of a solution that has recently been visited). However, cycles may still occur because the tabu lists have size limit. If a new entry enter the tabu list while it is full, the oldest entry will be removed (first in first out).

To intensify the search process, after a some iterations without finding a better candidate, we restart the search at the current best solution. And if after even more iterations without improving the best objective function, we restart the search from another solution acquired by the greedy algorithm.

Finally, the tabu search terminates when a certain stopping condition (e.g., maximal number of iterations reached) is satisfied.

Table 4 Parameters for tabu search, all numbers are rounded down to the nearest integer (floor)

	TL_{add} size	TL_{redis} size	$n_{reroute}$	$n_{reverse}$	n_{redis}
TSO	n/8	0	n/3	n/9	0
TSR	n/8	n/15	n/5	n/15	n/15

5 Computational Results

5.1 Instances

Since our problem is novel, no benchmarks exist in the literature for large-scale datasets specific to our context. As a result, we must adapt benchmark instances from related problems. In the classical RCPSP, the widely-used benchmark from [11] lacks information on transfer times between activities, which correspond to the travel times between tasks in our problem. Additionally, the largest instances in this benchmark only include 120 activities, which is in our opinion insufficient for modern warehouse settings. Although transfer times are considered in RCPSP instances with transfer time data from [12], the largest size in this set only involves 60 activities.

After carefully searching, we found that the Pick-up Delivery Problem data set in [13] is the most suitable for our problem to adapt. It is a well-known set of instances that are used intensively in PDP literature. For each problem size of [50, 100, 200, 300, 400, 500] tasks, the data set has 50 instances. The tasks in each instance in the original data already have information about the coordinates of the pick-up point and corresponding destination. To adapt to our problem, we first remove the unnecessary field and then add the processing time for each task, calculated by the Euclidean distance from the pickup point to the destination. For the number of robots required for each task, we generate a random number uniformly in the range [1, 5].

5.2 Numerical Results

We test our algorithms and formulation on a device equipped with Intel(R) Core(TM) i5- 8400 CPU @ 2.80GHz chipset. The MILP model and the heuristics were implemented using Python 3.9 and the formulation was solved using CPLEX 22.1.0.

We compare the tabu search with the addition of our neighborhood, called TSR from now, to the Tabu search with only two original neighbors, called TSO. The parameters for both algorithms are reported in Table 4. To keep the total solving time more competitive between both algorithms, in TSR, we reduce the number of neighbors to evaluate from reverse and reroute neighborhood i.e. In TSR, with the addition of the redistribute neighborhood, the algorithm explores fewer candidates from the two other ones. Both versions are run for 5000 iterations. Moreover, after some initial experiments, we decide to set the algorithm to restart from the best solution so far if an improving candidate could not be found in the last 300 iterations. If after 600 iterations without improvement, the algorithm restarts from a new initial solution.

Table 5 Integrated redistribute neighborhood into a tabu search

#ins	n	m	original	with redistribute neighborhood				
			time(s)	diff avg	diff min	diff max	time(s)	#better
10	100	10	663	2.35	-1.55	6.44	542	8
10	200	10	2152	1.21	-0.52	3.03	1997	8
10	300	10	3900	0.51	-2.28	1.16	4691	7
10	400	10	6972	1.71	-1.88	7.07	6734	9
10	500	10	11322	2.53	-5.82	8.58	11714	8
Average				1.66	-2.41	5.26		8

Table 5 evaluates the performance of TSR against TSO. Each row is the average of $\#ins$ instances of size n tasks and m robots. The *time* columns are solved time of each algorithm in seconds. The *diff* columns (average, min, max) indicate how much improvement in percentage of TSR compared to TSO in percentage on average, minimum, and maximum respectively. Finally, column *#better* indicates how many instances that TSR yields a better result than TSO.

In general, with the addition of our neighborhood, the tabu search consistently yields improvements in solution quality with an average of 8/10 cases for each instance size yielding better solution quality, with an improvement of 1.66 %. For smaller instances (100–200 tasks), average improvements range from 1.21% to 2.35%, with the TSR outperforming the original in 8 out of 10 cases. Notably, for 400-task instances, the average improvement rises to 1.71%, with the TSO achieving superior results in 9 out of 10 instances. However, while maximum improvements reach up to 8.58% (500 tasks), the algorithm does occasionally underperform, (e.g., -5.82% for instance with 500 tasks). The best improvements are also more significant (up to 8.58% with an average of 5.26%) than the cases of the worst improvements, generally under 2.3% with the exception of one case of -5.82%. Since we reduce the number of explored original neighborhoods in TSR, the solving time of both versions is comparable, even faster in TSR in smaller cases. For example on average, in the case of 100 TSR takes an average of 542 seconds vs 663 seconds in TSO, 1997 seconds in TSR vs 2152 seconds in TSO for instances with 200 tasks). In larger cases, the TSR run a little bit slower compared to TSO (11714 seconds for TSR vs 11322 seconds in TSO in instances with 500 tasks).

To gain more insight, we plot the progress of objective value relative to the initial solution over iterations of both versions of tabu search in Figure 12. The initial solution is the base value of 100%. Each line in the graph is the average relative objective value compared to the baseline of all tested instances, the blue curve is TSR and the orange one is TSO. While both algorithms decrease more steeply in the first few hundred iterations, suggesting the tabu search can quickly improve from an initial solution, the blue line is much steeper, which means TSR improves much faster initially. The blue curve maintains a lower objective value compared to the orange curve, indicating that TSR can find better consistently compared to TSO. The gap between the two curves widens as iterations progress, indicating the superior performance of the redistributing

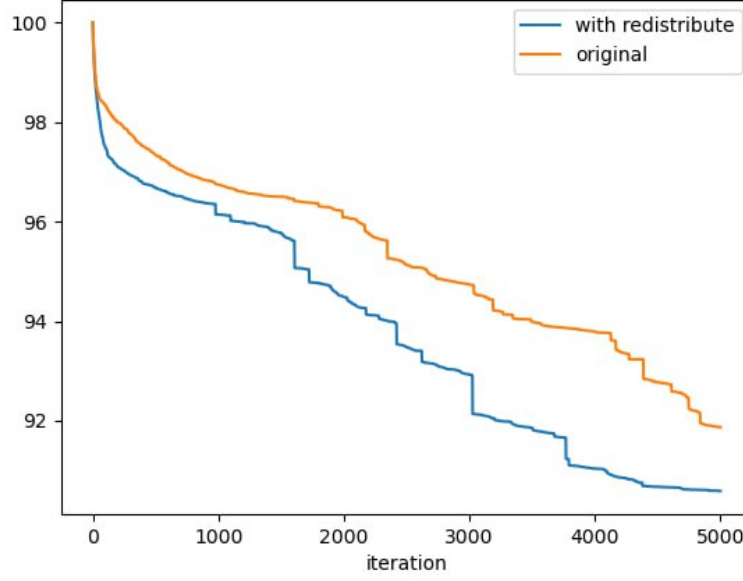


Fig. 12 Graph compares improvement between two original neighborhoods vs additional of redistribute neighborhood

tasks. We can observe several immediate steep decreases in the blue line, which doesn't appear as much in the original neighborhoods. It suggests that in TSR, candidates from the redistribute neighborhood help the algorithm escape local optima more effectively.

6 Conclusion

In this paper, we introduced the PDP-CR in more detail and proposed a sophisticated, effective yet natural neighborhood for PDPCR supported with illustrative examples. Besides theoretical results, we report numerical results to demonstrate that our proposed neighborhood enhances the metaheuristic's ability to explore more promising candidates, yielding superior solution quality while maintaining competitive solving times.

For further research, we could explore the more intricate version of the redistribute neighborhood where we could insert tasks in between the cut of topo-order and minimize the makespan at the same time, which we could apply to large neighborhood search algorithms or construct an initial solution for other algorithms. Furthermore, with appropriate modifications, the redistribute neighborhood can be adapted to other problems, such as RCPS and its variants with transfer times.

References

- [1] Toth, P., Vigo, D. (eds.): The Vehicle Routing Problem. SIAM monographs

on discrete mathematics and applications. Society for Industrial and Applied Mathematics, Philadelphia, Pa (2002)

- [2] Parragh, S.N., Doerner, K.F., Hartl, R.F.: A survey on pickup and delivery problems: Part II: Transportation between pickup and delivery locations. *Journal für Betriebswirtschaft* **58**(2), 81–117 (2008) <https://doi.org/10.1007/s11301-008-0036-4>
- [3] Berbeglia, G., Cordeau, J.-F., Laporte, G.: Dynamic pickup and delivery problems. *European Journal of Operational Research* **202**(1), 8–15 (2010) <https://doi.org/10.1016/j.ejor.2009.04.024>
- [4] Drexel, M.: Synchronization in Vehicle Routing—A Survey of VRPs with Multiple Synchronization Constraints. *Transportation Science* **46**(3), 297–316 (2012) <https://doi.org/10.1287/trsc.1110.0400>
- [5] Bredström, D., Rönnqvist, M.: Combined vehicle routing and scheduling with temporal precedence and synchronization constraints. *European Journal of Operational Research* **191**(1), 19–31 (2008) <https://doi.org/10.1016/j.ejor.2007.07.033>
- [6] Kim, B.-I., Koo, J., Park, J.: The combined manpower-vehicle routing problem for multi-staged services. *Expert Systems with Applications* **37**(12), 8424–8431 (2010) <https://doi.org/10.1016/j.eswa.2010.05.036>
- [7] Hollis, B.L., Forbes, M.A., Douglas, B.E.: Vehicle routing and crew scheduling for metropolitan mail distribution at Australia Post. *European Journal of Operational Research* **173**(1), 133–150 (2006) <https://doi.org/10.1016/j.ejor.2005.01.005>
- [8] Nguyen, C.T., Gayon, J.-P., Nguyen, V.H., Quilliot, A., Ta, A.S.: Pickup and Delivery Problem with Cooperative Robots. In: *Proceedings of the 12th International Symposium on Information and Communication Technology. SOICT '23*, pp. 695–700. Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3628797.3628988>
- [9] Nguyen, C.T., Gayon, J.-P., Nguyen, V.H., Ta, A.S.: A heuristics for Pickup-Delivery Problem with Cooperative Robots. In: *ICAMCS 2024 (International Conference on Applied Mathematics and Computer Science)*, Ha Noi, Vietnam (2024). International School – Vietnam National University Hanoi, Vietnam and Hanoi University of Science and Technology, Vietnam and Laboratoire de Conception, Optimisation et Modélisation des Systèmes, Université de Lorraine, France. <https://hal.science/hal-04991987>
- [10] Poppenborg, J., Knust, S.: A flow-based tabu search algorithm for the rcpsp with transfer times. *OR Spectrum* **38** (2015) <https://doi.org/10.1007/s00291-015-0402-2>

- [11] Kolisch, R., Sprecher, A.: PSPLIB - a project scheduling problem library: OR software - ORSEP operations research software exchange program. *European Journal of Operational Research* **96**(1), 205–216 (1997) [https://doi.org/10.1016/S0377-2217\(96\)00170-1](https://doi.org/10.1016/S0377-2217(96)00170-1)
- [12] Krüger, D., Scholl, A.: A heuristic solution framework for the resource constrained (multi-)project scheduling problem with sequence-dependent transfer times. *European Journal of Operational Research* **197**(2), 492–508 (2009) <https://doi.org/10.1016/j.ejor.2008.07.036>
- [13] Li, H., Lim, A.: A metaheuristic for the pickup and delivery problem with time windows. In: *Proceedings 13th IEEE International Conference on Tools with Artificial Intelligence. ICTAI 2001*, pp. 160–167 (2001). <https://doi.org/10.1109/ICTAI.2001.974461>